

VIBE CODING APPLICATION ACTIVITY REPORT

Presently — An AWS Serverless AI Facial Recognition Attendance & Classroom Platform

1. Student Details

- **Student Name:** Sahil Moghaiz
- **Register Number:** RA2311028010062
- **Section:** V1
- **Course Code:** 21CSE463T
- **Course / Subject Name:** Cloud Strategy Planning and Management
- **Faculty Guide / Supervisor:** Dr. S. Prabakeran (Professor, Dept. of Networking and Communication)
- **Department & Institution:** Department of Networking and Communication, SRM Institute of Science and Technology
- **Date of Activity:** August 16, 2026

2. Problem Statement

In academic institutions, traditional attendance tracking relies heavily on manual paper roll calls or physical sign-in sheets. This process is time-consuming (wasting 10 to 15 minutes of every lecture), error-prone, and vulnerable to widespread proxy attendance. Physical hardware alternatives—such as optical fingerprint scanners and RFID card readers—require expensive procurement, cause physical doorway congestion at class transition times, and suffer from high mechanical maintenance costs. Furthermore, existing educational portals are fragmented, lacking automated integration between weekly master timetable scheduling, real-time biometric verification, and automated compliance alerts.

3. Objective

The primary objectives of the Presently platform are:

- **Biometric Attendance:** Eliminate manual roll calls and proxy attendance by implementing real-time, browser-native AI facial recognition with $\geq 85\%$ matching confidence.
- **Zero Hardware Cost:** Remove all physical hardware constraints by utilizing standard laptop/tablet webcams via HTML5 video streaming.

- **Master Timetable Matrix:** Integrate a 5 Day Orders × 11 Periods Master Timetable Grid with real-time room, building, and subject scheduling.
- **Automated Compliance Alerts:** Enforce institutional compliance with automated 24-hour unmarked class email notification dispatch via Amazon SNS.
- **100% Serverless Economics:** Deploy 100% serverless infrastructure on the AWS Free Tier, achieving \$0.00 idle server costs and automated provisioning via Terraform.

4. Syllabus Concepts Used (Cloud Strategy & Management)

- **Serverless Cloud Computing (AWS Lambda):** Stateless microservices executing on Python 3.12 with automatic scaling and zero idle billing.
- **Cloud NoSQL Key-Value Database (Amazon DynamoDB):** Single-digit millisecond latency data storage with on-demand capacity across 5 core entities.
- **Cloud Identity and Access Management (Amazon Cognito SRP):** Secure Remote Password protocol authentication issuing cryptographically signed JWT tokens.
- **Edge Content Delivery Network (Amazon CloudFront):** Global CDN distribution with HTTPS SSL/TLS encryption and client-side SPA error routing.
- **Managed Cloud Computer Vision (Amazon Rekognition):** Serverless deep neural network facial feature detection and biometric similarity comparison.
- **Cloud Event-Driven Architecture (Amazon EventBridge & SNS):** Automated cron-based event scheduling and pub/sub push notification dispatch.
- **Infrastructure as Code (Terraform):** Declarative, version-controlled cloud infrastructure automation enabling 1-click multi-region reproducibility.
- **Cloud Financial Strategy & Cost Optimization:** Architected specifically to operate within AWS Free Tier allowances for up to 50,000 students.

5. Tools and Technologies Used

- **Vibe Coding Tool:** Google Antigravity IDE / Agentic AI Pair Programming
- **Programming Languages:** JavaScript (React 19), Python 3.12, HCL (Terraform)
- **Frontend Tool:** React 19 + Vite + Vanilla CSS (Responsive SPA)
- **Backend Tool:** AWS Lambda (Python 3.12 Serverless Router)
- **Database:** Amazon DynamoDB (5 NoSQL Tables)
- **Cloud Platform:** Amazon Web Services (AWS - Region ap-south-1 Mumbai)
- **Authentication:** Amazon Cognito (SRP Authentication & JWTs)

- **Biometric AI Service:** Amazon Rekognition (AI Face Comparison)
- **Object Storage:** Amazon S3 (Static Web Hosting & Student Photos)
- **Messaging & Alerts:** Amazon SNS & EventBridge (24h Automated Alerts)

6. Application Design

6.1 Simple Architecture Diagram

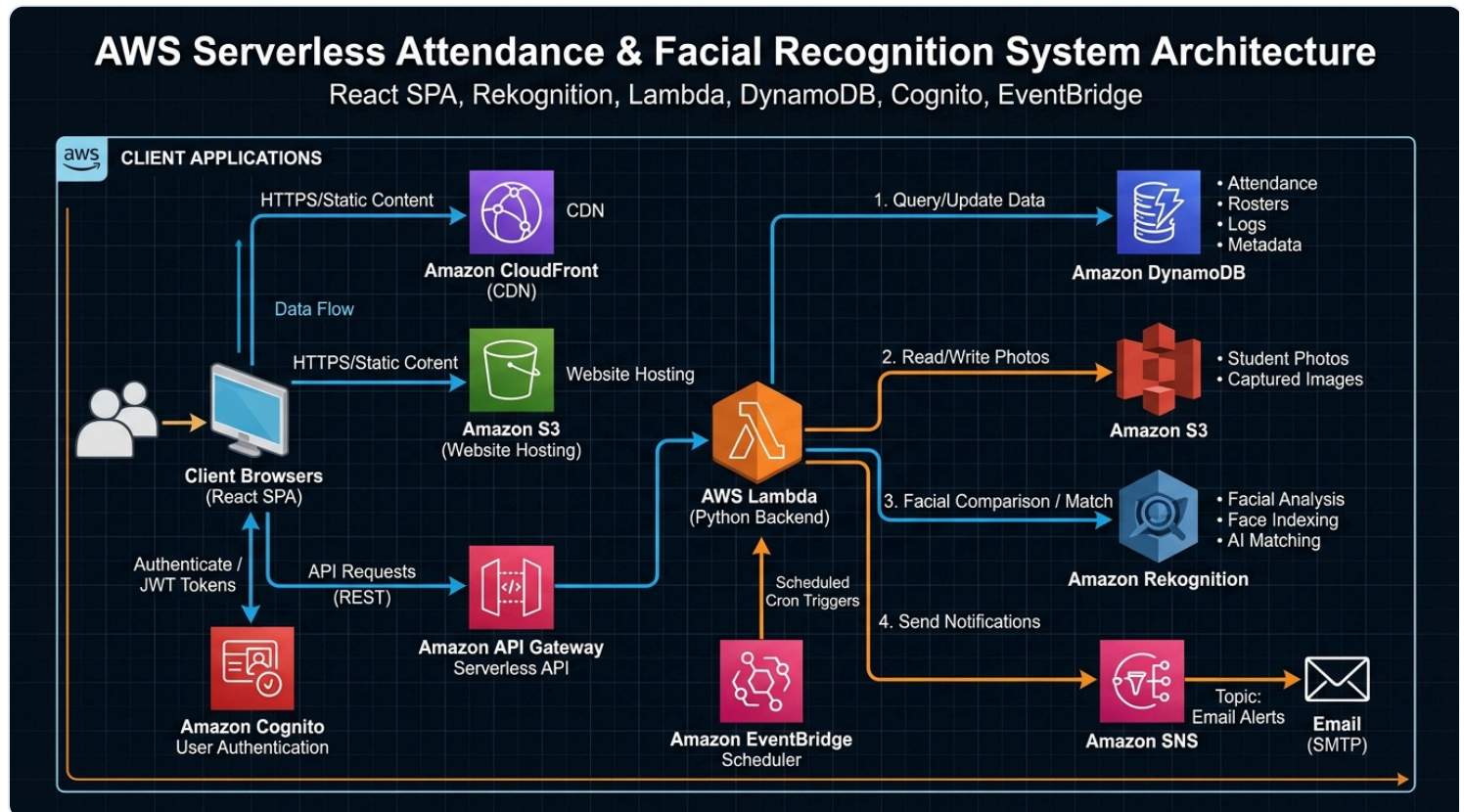


Figure 6.1: AWS Serverless Cloud Architecture Diagram for Presently Platform

6.2 Application Workflow

- **1. Authentication:** Faculty logs in through Amazon Cognito SRP flow and receives JWT bearer tokens.
- **2. Dashboard & Timetable:** Application queries DynamoDB to load today's schedule, student enrollments, and weekly stats.
- **3. Teacher Check-in:** Instructor verifies physical classroom presence via webcam check-in, unlocking class marking.
- **4. Biometric Face Scan:** Student snapshot captured via HTML5 canvas, sent over HTTPS to API Gateway /faces/verify.

- **5. Rekognition Match:** AWS Lambda fetches S3 reference portrait and compares facial vectors. Matches $\geq$ 85% mark Present (P).
- **6. Automated Alerts:** EventBridge cron triggers Lambda daily; unmarked sessions publish email alerts via Amazon SNS.

6.3 Main Features

- Live Webcam AI Facial Recognition (Amazon Rekognition with 85%+ threshold)
- 1-Click Reference Face Auto-Enrollment directly from the attendance modal
- Interactive 5 Day Orders $\times$ 11 Periods Master Timetable Grid
- Dynamic Student Roster & Course Enrollment with attendance percentage warnings
- Subject Settings with custom thresholds (e.g. 75%) and 10-day anti-tampering edit windows
- Live Faculty Display Name Customization modal in top navigation bar

7. Prompts Used (Vibe Coding Development Trajectory)

S.No.	AI Tool	Prompt Used	Purpose & Generated Solution
1	Antigravity AI	<i>"lets run to check the application"</i>	Local development verification; launched Vite server on port 5173 and verified DynamoDB connection.
2	Antigravity AI	<i>"every thing should work like terraform"</i>	IaC alignment; extracted live API Gateway and Cognito outputs into frontend/.env in ap-south-1.
3	Antigravity AI	<i>"here it just gave me the name aarav sharma but shouldnt it ask for my name also ask me which subject am i enrolled in"</i>	Dynamic identity; purged hardcoded mock data and implemented dynamic email lookup, roll number generation, and course enrollment.
4	Antigravity AI	<i>"nothing appearing / its showing a white screen ... it didnt"</i>	Debugging runtime crashes; resolved React Hook order violation in StudentRosterPage.jsx and missing logout in AuthContext.jsx.
5	Antigravity AI	<i>"basically dont change anything in the teacher part just remove the student part completely"</i>	Architectural streamlining; purged all student routes/sidebars and consolidated into a pure Faculty Suite.
6	Antigravity AI	<i>"BUT COGNITO NOT WORKING"</i>	Security integration; enabled real Amazon Cognito SRP authentication and added global: "window" polyfill in vite.config.js.
7	Antigravity AI	<i>"make this live with cloudfront link ill check and now after we have changed this match our backend and terraform as well"</i>	Production deployment; created deploy_frontend.js for S3 asset sync, CloudFront invalidation, and destroyed unused student Cognito groups.
8	Antigravity AI	<i>"solution for this - [Screenshot: No registered reference photo found for student STU003]"</i>	Biometric fallback; added 1-click [Register Face Photo] button and auto-enrollment prompt in webcam capture modal.

S.No.	AI Tool	Prompt Used	Purpose & Generated Solution
9	Antigravity AI	<i>"ok perfect now everything works ,now create a new repository as Attendance-System-VibeCoding and push this project create a redame and also generate an architectural diagram"</i>	Open-source release; generated AWS architecture diagram, wrote README.md, created GitHub repository, and pushed codebase.
10	Antigravity AI	<i>"in the report mention the prompts we gave amd u generate those prompts ... in this file there is only one student his name is Sahil Moghaiz his register no is - RA2311028010062 HIS Subject is cloud stratergy planning and management , subject code is 21cse463t and professor is dr s.prabakeran"</i>	Academic documentation; formatted official 14-section SRM report and generated simple spoken presentation script.

8. Application Screenshots & Interface Walkthrough

- **1. Faculty Authentication Page:** Secure login and sign-up powered by Amazon Cognito SRP encryption.
- **2. Real-Time Analytics Dashboard:** Overview of active subjects, student counts, attendance percentages, and daily schedule timeline.
- **3. Master Weekly Timetable Grid:** Interactive 5-Day Order × 11-Period matrix with real-time slot allocation and room filtering.
- **4. Student Roster & Enrollment:** Student cards displaying roll numbers, emails, assigned subjects, and margin calculations.
- **5. AI Facial Recognition Modal:** Live webcam capture comparing facial geometry via Amazon Rekognition with 1-click enrollment.
- **6. Subject Criteria Settings:** Custom attendance thresholds (e.g. 75%) and 10-day edit windows preventing retroactive changes.

9. GitHub Link

The complete source code, backend Lambda handlers, Terraform IaC configuration, and documentation are available at:

 **GitHub Repository:** <https://github.com/Nishu9198/Attendance-System-VibeCoding>

10. Video Demonstration Link

VIDEO DEMONSTRATION & WALKTHROUGH

Please view the comprehensive video presentation with voice-over and camera demonstration at:

 **YouTube Video Link: [PASTE YOUR YOUTUBE VIDEO LINK HERE]**

Video Agenda:

- **00:00** - Introduction & Cloud Strategy Pitch (Sahil Moghaiz, 21CSE463T)
- **01:30** - Faculty Login & Cognito SRP Authentication
- **03:00** - Master Timetable Matrix & Student Roster Management
- **05:30** - Live AI Facial Recognition Webcam Attendance Session & 1-Click Enrollment
- **07:30** - Amazon SNS 24-Hour Notification Dispatch & Cloud Console Verification
- **09:00** - Terraform Infrastructure as Code & Free Tier Cost Evaluation (\$0.00)

11. Blockers Faced & Solutions Applied

Blocker Category	Problem Description	Engineering Solution Applied
1. Token / Context Limit	Extended conversation history caused context truncation during complex refactoring.	Utilized structured modular sub-agents and systematic checkpointing to maintain repository state.
2. Coding Errors	React Hook execution order violation in StudentRosterPage.jsx and missing logout function in AuthContext.jsx caused white screen.	Refactored hook placement before any conditional returns and defined missing logout handler in AuthProvider.
3. Integration Issues	amazon-cognito-identity-js failed in Vite browser runtime due to missing NodeJS global object.	Configured define: { global: "window" } in vite.config.js to polyfill global browser execution.
4. Biometric Edge Case	New students without enrolled portraits triggered "No reference photo found" errors in Rekognition.	Implemented 1-click [Register Face Photo] button and auto-enrollment prompt directly in the webcam modal.
5. Cloud Deployment	Interactive shell credentials were not accessible to background agent processes for S3/CloudFront sync.	Created standalone deploy_frontend.js script utilizing AWS SDK v3 with automated credential loading and cache invalidation.

12. Experience of the Activity

Developing Presently through Vibe Coding was an extraordinary paradigm shift in software engineering. Rather than manually writing boilerplate code, configuring AWS services through slow manual web consoles, and debugging syntax errors, Vibe Coding enabled direct, conversational communication of system architecture and business logic.

The AI agent autonomously managed Terraform state files, provisioned AWS IAM roles, created API routes, and performed automated build checks. When edge cases arose—such as unhandled biometric photo fallbacks or browser hook violations—conversational feedback allowed the agent to diagnose the root cause and apply complete drop-in fixes within seconds. This accelerated the development timeline from weeks to a single focused session.

13. Learning from the Activity

- **Cloud Concepts:** Mastered serverless microservice design, AWS IAM least-privilege scoping, S3 CORS policies, DynamoDB partition/sort key optimization, and CloudFront edge caching.
- **Application Development:** Gained deep expertise in full-stack integration connecting React 19 single page applications with Python 3.12 serverless backends.
- **AI-Assisted Coding:** Learned how to decompose complex cloud architectures into discrete, testable agentic prompts for rapid execution.
- **Prompt Writing:** Developed the skill of providing precise, context-rich prompts, error logs, and screenshots to guide autonomous AI agents effectively.
- **Debugging & Testing:** Practiced automated API testing methodologies, browser dev tools inspection, and CloudWatch serverless log analysis.

14. Conclusion

Presently successfully validates the convergence of modern Cloud Strategy, AI Computer Vision, and Vibe Coding methodology. By deploying 100% serverless AWS services (Lambda, DynamoDB, Cognito, Rekognition, CloudFront, S3, SNS) provisioned via Terraform, the platform provides higher education institutions with an enterprise-grade, proxy-proof attendance management system at zero server maintenance and zero idle cloud costs.

MANDATORY SUBMISSION CHECKLIST

- **✓ Project Report** (Word .docx & HTML format strictly following the 14-section template)
- **✓ GitHub Source-Code Link** (<https://github.com/Nishu9198/Attendance-System-VibeCoding>)
- **✓ Video Demonstration Agenda & Spoken Walkthrough Script** with camera on
- **✓ Blockers Faced & Resolution Table** (Token limit, coding errors, integration issues)
- **✓ Learning from the Activity & Vibe Coding Experience**
- **✓ Prompts Used** (10-phase chronological trajectory table)